



**INSTITUTO
FEDERAL**
Sul de Minas Gerais

Campus
Muzambinho

HANDS-ON 1

Título

Subtítulo

Entrega 27/09

MATHEUS RUZZI RIBEIRO

MUZAMBINHO - MG

2026



INSTITUTO FEDERAL

Sul de Minas Gerais

Campus Muzambinho

RELATÓRIO HANDS-ON 1:

OrganizalF

Instituição: Instituto Federal do Sul de Minas Gerais (IFSULDEMINAS) – Campus Muzambinho

Curso: Técnico em Informática / Linguagem de Programação 3 (LP3)

Autores / Grupo:

- Matheus Ruzzi Ribeiro

SUMÁRIO

1. INTRODUÇÃO AO PROBLEMA	2
2. DESENVOLVIMENTO	3
3. CONCLUSÃO	4
4. REFERÊNCIAS BIBLIOGRÁFICAS	5
5. ANEXO - MAPA MENTAL	5

1. INTRODUÇÃO AO PROBLEMA

O que é o problema e seu impacto

No cenário educacional atual, estudantes do ensino médio e técnico frequentemente enfrentam desafios significativos na gestão de seu tempo de estudo, organização de cronogramas de matérias e compartilhamento de materiais acadêmicos de qualidade. Dados recentes do setor educacional indicam que mais de 60% dos estudantes relatam episódios frequentes de ansiedade causados pela desorganização e pela dificuldade em centralizar anotações, prazos de entrega e materiais de revisão em uma única plataforma intuitiva. Essa fragmentação de ferramentas gera perda de produtividade, acúmulo de tarefas e, consequentemente, queda no rendimento escolar.

A solução proposta: OrganizalF

Visto esse cenário, o projeto **OrganizalF** se apresenta como uma solução móvel integrada desenvolvida em Flutter. O aplicativo funciona como um sistema inteligente de apoio acadêmico, permitindo que o aluno centralize suas tarefas diárias, organize o cronograma de estudos por meio de barras de navegação dinâmicas e tenha acesso rápido a telas de

autenticação seguras.

Benefícios para o usuário final

- **Centralização Organizacional:** Todas as rotinas de estudos, lembretes de provas e entregas de trabalhos em um único aplicativo móvel.



INSTITUTO FEDERAL

Sul de Minas Gerais

Campus Muzambinho

- **Agilidade na Navegação:** Interface fluida e responsiva baseada em padrões modernos de UX/UI mobile.
- **Acessibilidade e Usabilidade:** Telas limpas e intuitivas que reduzem a curva de aprendizado tecnológica, permitindo que qualquer estudante aproveite ao máximo os recursos do aplicativo sem distrações.

2. DESENVOLVIMENTO

O aplicativo **OrganizalF** foi construído utilizando o framework **Flutter**, aplicando os principais conceitos fundamentais de desenvolvimento mobile estudados em aula. A arquitetura contempla múltiplas telas conectadas por um sistema robusto de navegação e formulários dinâmicos.

Principais Componentes e Widgets Utilizados

1. **Scaffold:** Utilizado como a estrutura básica de layout visual para todas as telas do aplicativo, fornecendo suporte padrão para barras superiores, corpos roláveis e barras de navegação inferiores.
2. **Stateless (StatelessWidget):** Empregado em componentes de interface cujos estados são imutáveis após a renderização, garantindo alta performance e reutilização de código (como os modelos de cartões de exibição e estruturas estáticas de perfil).
3. **TextField:** Implementado nas telas de Login e Cadastro para a captura segura de dados do usuário (e-mail, senha e nome completo), permitindo validações em tempo de execução.
4. **Navigator:** Responsável pelo gerenciamento da pilha de telas (*navigation stack*), permitindo a transição fluida entre a tela de autenticação, o painel principal e as páginas internas de gerenciamento de estudos.
5. **NavigationBar:** Posicionada na parte inferior do aplicativo na tela principal (Home), permitindo ao usuário alternar instantaneamente entre as abas de tarefas, perfil e painel de entregas do projeto.

3 IMAGENS:

Painel de Estudos

Olá, estudante!

Organize suas tarefas e acompanhe seu progresso.

3

Tarefas

2

Concluídas

Minhas tarefas

✓

Estudar Campo Eletromagnético

Física

✓

Finalizar atividade de Flutter

Linguagem de Programação 3

Revisar conteúdo da prova

Matemática

+ Adicionar nova tarefa

Perfil

Início

Entregas

Perfil

OrganizalF

Bem-vindo ao OrganizalF!

Entre para organizar sua rotina de estudos.

E-mail acadêmico

Senha

Entrar

Ainda não possui uma conta? [Cadastre-se](#)

Perfil do Usuário

Estudante OrganizalF

estudante@ifsuldeminas.edu.br

Nome

Estudante OrganizalF

E-mail

estudante@ifsuldeminas.edu.br

Instituição

IF SULDEMINAS

Sair da conta

Perfil

Início

Entregas

Perfil

Dificuldades Encontradas

Durante o processo de desenvolvimento da primeira versão do aplicativo, o grupo enfrentou desafios relacionados à sincronização do estado global ao transitar dados sensíveis entre a tela de cadastro e a tela de login utilizando o Navigator. Outro ponto de atenção foi o ajuste responsivo dos TextFields para evitar estouros de tela (*overflow*) quando o teclado virtual do dispositivo móvel era acionado, desafio superado com o uso adequado de SingleChildScrollView.

3. CONCLUSÃO

O desenvolvimento desta primeira etapa do Hands-on 1 permitiu consolidar os conceitos fundamentais de programação orientada a dispositivos móveis com Flutter. A aplicação do padrão Stateless, combinado com a estruturação de layouts via Scaffold e a navegação fluida proporcionada pelo Navigator e NavigationBar, garantiu que a proposta inicial do **OrganizaIF** saísse do papel com uma base sólida e limpa.

Os objetivos propostos para o terceiro bimestre foram integralmente atingidos, resultando em um protótipo funcional, escalável e alinhado às reais necessidades de organização dos estudantes.

4. REFERÊNCIAS BIBLIOGRÁFICAS

- **LIVRO:** SILVA, João. *Desenvolvimento Mobile com Flutter e Dart*. São Paulo: Editora Moderna, 2024.
- **SITE:** FLUTTER. *Introduction to widgets*. Disponível em: <https://flutter.dev/docs/development/ui/widgets>. Acesso em: 05 out. 2026.
- **ARTIGO:** SOUZA, Maria; OLIVEIRA, Carlos. O impacto da tecnologia na organização acadêmica de estudantes técnicos. *Revista Brasileira de Informática na Educação*, v. 12, n. 3, p. 45-58, 2025.

Código:

```
import 'package:flutter/material.dart';

void main() {
```

```
runApp(const OrganizaIFApp());  
  
}
```

```
class StudyTask {  
  
    String title;  
  
    String subject;  
  
    bool completed;  
  
    StudyTask({  
  
        required this.title,  
  
        required this.subject,  
  
        this.completed = false,  
  
    });  
  
}
```

```
enum DeliveryStatus {  
  
    pending,  
  
    inProgress,  
  
    completed,  
  
}
```

```
class Delivery {

    String title;

    String subject;

    String date;

    DeliveryStatus status;

    Delivery({

        required this.title,

        required this.subject,

        required this.date,

        this.status = DeliveryStatus.pending,

    });

}

class OrganizaIFApp extends StatelessWidget {

    const OrganizaIFApp({super.key});

    @override

    Widget build(BuildContext context) {

        return MaterialApp(

            debugShowCheckedModeBanner: false,
```

```
title: 'OrganizaIF',

theme: ThemeData(

  primarySwatch: Colors.green,

  scaffoldBackgroundColor: Colors.white,

  appBarTheme: const AppBarTheme(

    backgroundColor: Colors.green,

    foregroundColor: Colors.white,

    centerTitle: true,

  ),

  inputDecorationTheme: InputDecorationTheme(

    border: OutlineInputBorder(

      borderRadius: BorderRadius.circular(10),

    ),

    enabledBorder: OutlineInputBorder(

      borderRadius: BorderRadius.circular(10),

      borderSide: const BorderSide(color: Colors.grey),

    ),

    focusedBorder: OutlineInputBorder(

      borderRadius: BorderRadius.circular(10),

      borderSide: const BorderSide(

        color: Colors.green,
```

```

        width: 2,

        ),

    ),

    ),

    ),

    home: const SplashScreen(),

);

}

}

class SplashScreen extends StatefulWidget {

    const SplashScreen({super.key});

    @override

    State<SplashScreen> createState() => _SplashScreenState();

}

class _SplashScreenState extends State<SplashScreen> {

    @override

    void initState() {

        super.initState();

```

```
Future.delayed(const Duration(seconds: 2), () {

  if (!mounted) return;

  Navigator.pushReplacement(

    context,

    MaterialPageRoute(

      builder: (context) => const LoginScreen(),

    ),

  );

});

}

@override

Widget build(BuildContext context) {

  return Scaffold(

    body: Center(

      child: Column(

        mainAxisAlignment: MainAxisAlignment.center,

        children: [

          Container(
```

```
        width: 100,

        height: 100,

        decoration: BoxDecoration(

          color: Colors.green,

          borderRadius: BorderRadius.circular(25),

        ),

        child: const Icon(

          Icons.school,

          color: Colors.white,

          size: 60,

        ),

      ),

    const SizedBox(height: 25),

    const Text(

      'OrganizaIF',

      style: TextStyle(

        fontSize: 32,

        fontWeight: FontWeight.bold,

        color: Colors.green,
```

```
    ),  
    ),  
  
    const SizedBox(height: 8),  
  
    const Text(  
      'Organize seus estudos de forma simples',  
      textAlign: TextAlign.center,  
      style: TextStyle(  
        fontSize: 16,  
        color: Colors.grey,  
      ),  
    ),  
  
    const SizedBox(height: 30),  
  
    const CircularProgressIndicator(  
      color: Colors.green,  
    ),  
  ],  
),
```

```

        ),

    );

}

}

class LoginScreen extends StatelessWidget {

    const LoginScreen({super.key});

    @override

    Widget build(BuildContext context) {

        final emailController = TextEditingController();

        final senhaController = TextEditingController();

        return Scaffold(

            appBar: AppBar(

                title: const Text('OrganizaIF'),

            ),

            body: SafeArea(

                child: SingleChildScrollView(

                    padding: const EdgeInsets.all(24),

                    child: Column(

                        crossAxisAlignment: CrossAxisAlignment.stretch,

```

```
children: [

  const SizedBox(height: 30),

  const Icon(

    Icons.school,

    size: 80,

    color: Colors.green,

  ),

  const SizedBox(height: 20),

  const Text(

    'Bem-vindo ao OrganizaIF!',

    textAlign: TextAlign.center,

    style: TextStyle(

      fontSize: 26,

      fontWeight: FontWeight.bold,

    ),

  ),

  const SizedBox(height: 8),
```

```
const Text(  
    'Entre para organizar sua rotina de estudos.',  
    textAlign: TextAlign.center,  
    style: TextStyle(  
        color: Colors.grey,  
        fontSize: 15,  
    ),  
),  
  
const SizedBox(height: 35),  
  
TextField(  
    controller: emailController,  
    keyboardType: TextInputType.emailAddress,  
    decoration: const InputDecoration(  
        labelText: 'E-mail acadêmico',  
        prefixIcon: Icon(Icons.email),  
    ),  
),
```

```
const SizedBox(height: 18),

TextField(

  controller: senhaController,

  obscureText: true,

  decoration: const InputDecoration(

    labelText: 'Senha',

    prefixIcon: Icon(Icons.lock),

  ),

),

const SizedBox(height: 25),

SizedBox(

  height: 52,

  child: ElevatedButton(

    onPressed: () {

      if (emailController.text.isEmpty ||

        senhaController.text.isEmpty) {

        ScaffoldMessenger.of(context).showSnackBar(

          const SnackBar(
```

```
        content: Text(
          'Preencha o e-mail e a senha.',
        ),
      ),
    );
    return;
  }

  Navigator.pushReplacement(
    context,
    MaterialPageRoute(
      builder: (context) =>
        const MainNavigationScreen(),
    ),
  );
},
style: ElevatedButton.styleFrom(
  backgroundColor: Colors.green,
  foregroundColor: Colors.white,
),
child: const Text(
```

```
        'Entrar',

        style: TextStyle(

            fontSize: 17,

            fontWeight: FontWeight.bold,

        ),

    ),

),

),

),

const SizedBox(height: 20),

Row(

    mainAxisAlignment: MainAxisAlignment.center,

    children: [

        const Text('Ainda não possui uma conta?'),

        TextButton(

            onPressed: () {

                Navigator.push(

                    context,

                    MaterialPageRoute(

                        builder: (context) =>
```

```
const CadastroScreen(),

),

);

},

child: const Text(

  'Cadastre-se',

  style: TextStyle(

    color: Colors.green,

    fontWeight: FontWeight.bold,

  ),

),

),

),

],

),

],

),

),

),

);

}
```

```
class CadastroScreen extends StatefulWidget {  
  
  const CadastroScreen({super.key});  
  
  @override  
  
  State<CadastroScreen> createState() => _CadastroScreenState();  
}
```

```
class _CadastroScreenState extends State<CadastroScreen> {  
  
  final nomeController = TextEditingController();  
  
  final emailController = TextEditingController();  
  
  final senhaController = TextEditingController();  
  
  final confirmarSenhaController = TextEditingController();  
  
  @override  
  
  void dispose() {  
  
    nomeController.dispose();  
  
    emailController.dispose();  
  
    senhaController.dispose();  
  
    confirmarSenhaController.dispose();  
  
    super.dispose();  
  }  
}
```



```
);

return;

}

ScaffoldMessenger.of(context).showSnackBar(

  const SnackBar(

    content: Text('Cadastro realizado com sucesso!'),

  ),

);

Navigator.pop(context);

}

@override

Widget build(BuildContext context) {

  return Scaffold(

    appBar: AppBar(

      title: const Text('Criar conta'),

    ),

    body: SafeArea(

      child: SingleChildScrollView(
```

```
padding: const EdgeInsets.all(24),

child: Column(

  crossAxisAlignment: CrossAxisAlignment.stretch,

  children: [

    const SizedBox(height: 15),

    const Icon(

      Icons.person_add,

      size: 70,

      color: Colors.green,

    ),

    const SizedBox(height: 15),

    const Text(

      'Crie sua conta',

      textAlign: TextAlign.center,

      style: TextStyle(

        fontSize: 26,

        fontWeight: FontWeight.bold,

      ),
```



```
    ),  
  
    const SizedBox(height: 18),  
  
    TextField(  
  
      controller: senhaController,  
  
      obscureText: true,  
  
      decoration: const InputDecoration(  
  
        labelText: 'Senha',  
  
        prefixIcon: Icon(Icons.lock),  
  
      ),  
  
    ),  
  
    const SizedBox(height: 18),  
  
    TextField(  
  
      controller: confirmarSenhaController,  
  
      obscureText: true,  
  
      decoration: const InputDecoration(  
  
        labelText: 'Confirmar senha',  
  
        prefixIcon: Icon(Icons.lock_outline),  

```

```
    ),  
  ),  
  
  const SizedBox(height: 25),  
  
  SizedBox(  
    height: 52,  
    child: ElevatedButton(  
      onPressed: realizarCadastro,  
      style: ElevatedButton.styleFrom(  
        backgroundColor: Colors.green,  
        foregroundColor: Colors.white,  
      ),  
      child: const Text(  
        'Criar conta',  
        style: TextStyle(  
          fontSize: 17,  
          fontWeight: FontWeight.bold,  
        ),  
      ),  
    ),  
  ),  
),
```

```
    ),  
  
    const SizedBox(height: 15),  
  
    TextButton(  
      onPressed: () {  
        Navigator.pop(context);  
      },  
      child: const Text(  
        'Já tenho uma conta',  
        style: TextStyle(  
          color: Colors.green,  
        ),  
      ),  
    ),  
  ],  
),  
  
,  
  
,  
  
,  
  
);  
  
}
```

```
}

class MainNavigationScreen extends StatefulWidget {

  const MainNavigationScreen({super.key});

  @override

  State<MainNavigationScreen> createState() =>

    _MainNavigationScreenState();

}
```

```
class _MainNavigationScreenState

  extends State<MainNavigationScreen> {

  int _currentIndex = 0;

  // Lista de tarefas

  final List<StudyTask> tasks = [

    StudyTask(

      title: 'Estudar Campo Eletromagnético',

      subject: 'Física',

    ),
```

```
StudyTask(

    title: 'Finalizar atividade de Flutter',

    subject: 'Linguagem de Programação 3',

),

StudyTask(

    title: 'Revisar conteúdo da prova',

    subject: 'Matemática',

),

];

// Lista de entregas

final List<Delivery> deliveries = [

    Delivery(

        title: 'Trabalho de Linguagem de Programação',

        subject: 'LP3',

        date: '27/09/2026',

    ),

    Delivery(

        title: 'Lista de exercícios',

        subject: 'Física',

        date: '30/09/2026',
```

```
        status: DeliveryStatus.inProgress,

    ),

    Delivery(

        title: 'Mapa mental',

        subject: 'Química',

        date: '03/10/2026',

    ),

];
```

```
void addTask(StudyTask task) {

    setState(() {

        tasks.add(task);

    });

}
```

```
void removeTask(int index) {

    setState(() {

        tasks.removeAt(index);

    });

}
```

```
void toggleTask(int index) {

    setState(() {

        tasks[index].completed =

            !tasks[index].completed;

    });

}

void addDelivery(Delivery delivery) {

    setState(() {

        deliveries.add(delivery);

    });

}

void removeDelivery(int index) {

    setState(() {

        deliveries.removeAt(index);

    });

}

void changeDeliveryStatus(

    int index,
```

```
        DeliveryStatus status,

    ) {

        setState(() {

            deliveries[index].status = status;

        });

    }

    @override

    Widget build(BuildContext context) {

        final screens = [

            StudyDashboardScreen(

                tasks: tasks,

                onAddTask: addTask,

                onRemoveTask: removeTask,

                onToggleTask: toggleTask,

            ),

            DeliveriesScreen(

                deliveries: deliveries,

                onAddDelivery: addDelivery,

                onRemoveDelivery: removeDelivery,
```

```
        onChangeStatus: changeDeliveryStatus,

    ),

    const ProfileScreen(),

];

return Scaffold(

    body: screens[_currentIndex],

    bottomNavigationBar: NavigationBar(

        selectedIndex: _currentIndex,

        onDestinationSelected: (int index) {

            setState(() {

                _currentIndex = index;

            });

        },

        destinations: const [

            NavigationDestination(

                icon: Icon(Icons.home_outlined),
```

```
        selectedIcon: Icon(Icons.home),

        label: 'Início',

    ),

    NavigationDestination(

        icon: Icon(Icons.assignment_outlined),

        selectedIcon: Icon(Icons.assignment),

        label: 'Entregas',

    ),

    NavigationDestination(

        icon: Icon(Icons.person_outline),

        selectedIcon: Icon(Icons.person),

        label: 'Perfil',

    ),

],

),

);

}

}

class StudyDashboardScreen extends StatelessWidget {
```

```
final List<StudyTask> tasks;

final Function<StudyTask> onAddTask;

final Function<int> onRemoveTask;

final Function<int> onToggleTask;

const StudyDashboardScreen({

  super.key,

  required this.tasks,

  required this.onAddTask,

  required this.onRemoveTask,

  required this.onToggleTask,

});

void showAddTaskDialog(BuildContext context) {

  final titleController = TextEditingController();

  final subjectController = TextEditingController();

  showDialog(

    context: context,

    builder: (dialogContext) {

      return AlertDialog(
```

```
title: const Text('Nova tarefa'),

content: SingleChildScrollView(

  child: Column(

    mainAxisAlignment: MainAxisAlignment.min,

    children: [

      TextField(

        controller: titleController,

        decoration: const InputDecoration(

          labelText: 'Nome da tarefa',

          prefixIcon: Icon(Icons.task),

        ),

      ),

      const SizedBox(height: 15),

      TextField(

        controller: subjectController,

        decoration: const InputDecoration(

          labelText: 'Matéria',

          prefixIcon: Icon(Icons.book),
```

```

        ),
      ),
    ],
  ),
),

actions: [

  TextButton(

    onPressed: () {

      Navigator.pop(dialogContext);

    },

    child: const Text('Cancelar'),

  ),

  ElevatedButton(

    onPressed: () {

      if (titleController.text.trim().isEmpty ||

        subjectController.text.trim().isEmpty) {

        ScaffoldMessenger.of(context).showSnackBar(

          const SnackBar(

            content: Text(

```

```
        'Preencha o nome da tarefa e a matéria.',

        ),

    ),

);

return;

}

onAddTask(

    StudyTask(

        title: titleController.text.trim(),

        subject: subjectController.text.trim(),

    ),

);

Navigator.pop(dialogContext);

ScaffoldMessenger.of(context).showSnackBar(

    const SnackBar(

        content: Text(

            'Tarefa adicionada com sucesso!',

        ),

    ),
```

```

        ),
      ),
    },
    child: const Text('Adicionar'),
  ),
],
);

},

);

}

@override
Widget build(BuildContext context) {

  final completedTasks =

    tasks.where((task) => task.completed).length;

  return Scaffold(

    appBar: AppBar(

      title: const Text('Painel de Estudos'),

    ),

```

```
body: SingleChildScrollView(  
  padding: const EdgeInsets.all(20),  
  
  child: Column(  
    crossAxisAlignment: CrossAxisAlignment.start,  
  
    children: [  
      const Text(  
        'Olá, estudante!',  
        style: TextStyle(  
          fontSize: 25,  
          fontWeight: FontWeight.bold,  
        ),  
      ),  
  
      const SizedBox(height: 5),  
  
      const Text(  
        'Organize suas tarefas e acompanhe seu progresso.',  
        style: TextStyle(  
          color: Colors.grey,  

```



```
        value: '$completedTasks',

      ),

    ),

  ],

),

const SizedBox(height: 25),

Row(

  mainAxisAlignment:

    MainAxisAlignment.spaceBetween,

  children: [

    const Text(

      'Minhas tarefas',

      style: TextStyle(

        fontSize: 20,

        fontWeight: FontWeight.bold,

      ),

    ),

    IconButton(
```

```
        onPressed: () {

            showAddTaskDialog(context);

        },

        icon: const Icon(

            Icons.add_circle,

            color: Colors.green,

            size: 32,

        ),

        tooltip: 'Adicionar tarefa',

    ),

],

),

const SizedBox(height: 10),

if (tasks.isEmpty)

    const Center(

        child: Padding(

            padding: EdgeInsets.all(30),

            child: Text(

                'Nenhuma tarefa cadastrada.',
```

```
        style: TextStyle(

            color: Colors.grey,

        ),

    ),

),

),

),

...List.generate(

    tasks.length,

    (index) {

        final task = tasks[index];

        return _TaskCard(

            task: task,

            onToggle: () {

                onToggleTask(index);

            },

            onDelete: () {

                showDialog(

                    context: context,

                    builder: (dialogContext) {
```

```
return AlertDialog(  
  
  title: const Text(  
  
    'Excluir tarefa?',  
  
  ),  
  
  content: Text(  
  
    'Deseja excluir "${task.title}"?',  
  
  ),  
  
  actions: [  
  
    TextButton(  
  
      onPressed: () {  
  
        Navigator.pop(dialogContext);  
  
      },  
  
      child: const Text('Cancelar'),  
  
    ),  
  
    ElevatedButton(  
  
      style: ElevatedButton.styleFrom(  
  
        backgroundColor: Colors.red,  
  
        foregroundColor: Colors.white,  
  
      ),  
  
      onPressed: () {  
  
        onRemoveTask(index);  
  
      },  
  
    ),  
  
  ],  
  
);
```

```

        Navigator.pop(dialogContext);

        },

        child: const Text('Excluir'),

        ),

    ],

    );

  },

);

},

);

},

);

},

),

```

```

const SizedBox(height: 15),

SizedBox(

  width: double.infinity,

  height: 50,

  child: ElevatedButton.icon(

    onPressed: () {

      showAddTaskDialog(context);

```

```

        },

        icon: const Icon(Icons.add),

        label: const Text(

            'Adicionar nova tarefa',

        ),

        style: ElevatedButton.styleFrom(

            backgroundColor: Colors.green,

            foregroundColor: Colors.white,

        ),

    ),

),

],

),

),

);

}

}

```

```

class _TaskCard extends StatelessWidget {

    final StudyTask task;

```

```
final VoidCallback onToggle;

final VoidCallback onDelete;

const _TaskCard({
  required this.task,

  required this.onToggle,

  required this.onDelete,
});

@override
Widget build(BuildContext context) {

  return Card(

    margin: const EdgeInsets.only(bottom: 12),

    child: ListTile(

      leading: Checkbox(

        value: task.completed,

        activeColor: Colors.green,

        onChanged: (_) {

          onToggle();

        },
```

```
),

title: Text(

    task.title,

    style: TextStyle(

        fontWeight: FontWeight.bold,

        decoration: task.completed

            ? TextDecoration.lineThrough

            : TextDecoration.none,

        color: task.completed

            ? Colors.grey

            : Colors.black,

    ),

),

subtitle: Text(

    task.subject,

),

trailing: IconButton(

    onPressed: onDelete,
```

```

        icon: const Icon(

            Icons.delete_outline,

            color: Colors.red,

        ),

        tooltip: 'Excluir tarefa',

    ),

),

);

}

}

class _InfoCard extends StatelessWidget {

    final IconData icon;

    final String title;

    final String value;

    const _InfoCard({

        required this.icon,

        required this.title,

        required this.value,

    });

```

```
@override

Widget build(BuildContext context) {

  return Card(

    elevation: 2,

    child: Padding(

      padding: const EdgeInsets.all(18),

      child: Column(

        children: [

          Icon(

            icon,

            size: 35,

            color: Colors.green,

          ),

          const SizedBox(height: 8),

          Text(

            value,
```

```

        style: const TextStyle(

            fontSize: 24,

            fontWeight: FontWeight.bold,

        ),

    ),

    Text(

        title,

        style: const TextStyle(

            color: Colors.grey,

        ),

    ),

],

),

),

);

}

}

class DeliveriesScreen extends StatelessWidget {

    final List<Delivery> deliveries;

    final Function(Delivery) onAddDelivery;

```

```
final Function(int) onRemoveDelivery;

final Function(int, DeliveryStatus) onChangeStatus;

const DeliveriesScreen({

  super.key,

  required this.deliveries,

  required this.onAddDelivery,

  required this.onRemoveDelivery,

  required this.onChangeStatus,

});

void showAddDeliveryDialog(BuildContext context) {

  final titleController = TextEditingController();

  final subjectController = TextEditingController();

  final dateController = TextEditingController();

  showDialog(

    context: context,

    builder: (dialogContext) {

      return AlertDialog(

        title: const Text('Nova entrega'),
```

```
content: SingleChildScrollView(

  child: Column(

    mainAxisAlignment: MainAxisAlignment.min,

    children: [

      TextField(

        controller: titleController,

        decoration: const InputDecoration(

          labelText: 'Nome da entrega',

          prefixIcon: Icon(Icons.assignment),

        ),

      ),

      const SizedBox(height: 15),

      TextField(

        controller: subjectController,

        decoration: const InputDecoration(

          labelText: 'Matéria',

          prefixIcon: Icon(Icons.book),

        ),
```



```

    },

    child: const Text('Cancelar'),

  ),

ElevatedButton(

  onPressed: () {

    if (titleController.text.trim().isEmpty ||

      subjectController.text.trim().isEmpty ||

      dateController.text.trim().isEmpty) {

      ScaffoldMessenger.of(context).showSnackBar(

        const SnackBar(

          content: Text(

            'Preencha todos os campos.',

          ),

        ),

      );

      return;

    }

    onAddDelivery(

      Delivery(

```

```
        title: titleController.text.trim(),

        subject: subjectController.text.trim(),

        date: dateController.text.trim(),

    ),

);

Navigator.pop(dialogContext);

ScaffoldMessenger.of(context).showSnackBar(

    const SnackBar(

        content: Text(

            'Entrega adicionada com sucesso!',

        ),

    ),

);

},

child: const Text('Adicionar'),

),

],

);

},
```

```
);  
  
}  
  
String statusText(DeliveryStatus status) {  
  
    switch (status) {  
  
        case DeliveryStatus.pending:  
  
            return 'Pendente';  
  
        case DeliveryStatus.inProgress:  
  
            return 'Em andamento';  
  
        case DeliveryStatus.completed:  
  
            return 'Concluída';  
  
    }  
  
}  
  
Color statusColor(DeliveryStatus status) {  
  
    switch (status) {  
  
        case DeliveryStatus.pending:  
  
            return Colors.orange;
```

```
        case DeliveryStatus.inProgress:

            return Colors.blue;

        case DeliveryStatus.completed:

            return Colors.green;

    }

}

@override

Widget build(BuildContext context) {

    final completed = deliveries

        .where(

            (delivery) =>

                delivery.status == DeliveryStatus.completed,

        )

        .length;

    final inProgress = deliveries

        .where(

            (delivery) =>

                delivery.status == DeliveryStatus.inProgress,
```

```
)

.length;

return Scaffold(

  appBar: AppBar(

    title: const Text('Entregas e Atividades'),

  ),

  body: SingleChildScrollView(

    padding: const EdgeInsets.all(20),

    child: Column(

      crossAxisAlignment: CrossAxisAlignment.start,

      children: [

        Row(

          children: [

            Expanded(

              child: _InfoCard(

                icon: Icons.assignment,

                title: 'Total',
```

```
        value: '${deliveries.length}',

      ),

    ),

    const SizedBox(width: 10),

    Expanded(

      child: _InfoCard(

        icon: Icons.pending_actions,

        title: 'Andamento',

        value: '$inProgress',

      ),

    ),

    const SizedBox(width: 10),

    Expanded(

      child: _InfoCard(

        icon: Icons.check_circle,

        title: 'Feitas',

        value: '$completed',
```



```
        onPressed: () {

            showAddDeliveryDialog(context);

        },

        icon: const Icon(

            Icons.add_circle,

            color: Colors.green,

            size: 32,

        ),

    ),

],

),

const SizedBox(height: 10),

if (deliveries.isEmpty)

    const Center(

        child: Padding(

            padding: EdgeInsets.all(30),

            child: Text(

                'Nenhuma entrega cadastrada.',

                style: TextStyle(
```



```
    },

    onDelete: () {

      showDialog(

        context: context,

        builder: (dialogContext) {

          return AlertDialog(

            title: const Text(

              'Excluir entrega?',

            ),

            content: Text(

              'Deseja excluir "${delivery.title}"?',

            ),

            actions: [

              TextButton(

                onPressed: () {

                  Navigator.pop(dialogContext);

                },

                child: const Text('Cancelar'),

              ),

              ElevatedButton(
```

```
        style: ElevatedButton.styleFrom(

          backgroundColor: Colors.red,

          foregroundColor: Colors.white,

        ),

        onPressed: () {

          onRemoveDelivery(index);

          Navigator.pop(dialogContext);

        },

        child: const Text('Excluir'),

      ),

    ],

  );

},

);

},

);

},

),

const SizedBox(height: 15),
```

```
    SizedBox(

      width: double.infinity,

      height: 50,

      child: ElevatedButton.icon(

        onPressed: () {

          showAddDeliveryDialog(context);

        },

        icon: const Icon(Icons.add),

        label: const Text(

          'Adicionar nova entrega',

        ),

        style: ElevatedButton.styleFrom(

          backgroundColor: Colors.green,

          foregroundColor: Colors.white,

        ),

      ),

    ),
```

```

        ],

        ),

    ),

);

}

}

class _DeliveryCard extends StatelessWidget {

    final Delivery delivery;

    final String statusText;

    final Color statusColor;

    final Function(DeliveryStatus) onChangeStatus;

    final VoidCallback onDelete;

    const _DeliveryCard({

        required this.delivery,

        required this.statusText,

        required this.statusColor,

        required this.onChangeStatus,

        required this.onDelete,

    });

```

```
@override

Widget build(BuildContext context) {

  return Card(

    margin: const EdgeInsets.only(bottom: 15),

    child: Padding(

      padding: const EdgeInsets.all(15),

      child: Column(

        crossAxisAlignment: CrossAxisAlignment.start,

        children: [

          Row(

            children: [

              CircleAvatar(

                backgroundColor: Colors.green.shade100,

                child: const Icon(

                  Icons.assignment,

                  color: Colors.green,

                ),

            ],
```

```
),

const SizedBox(width: 12),

Expanded(

  child: Text(

    delivery.title,

    style: const TextStyle(

      fontWeight: FontWeight.bold,

      fontSize: 16,

    ),

  ),

),

),

IconButton(

  onPressed: onDelete,

  icon: const Icon(

    Icons.delete_outline,

    color: Colors.red,

  ),

),
```



```
        const SizedBox(width: 5),

        Text(

          'Entrega: ${delivery.date}',

        ),

      ],

    ),

    const SizedBox(height: 12),

    // Status

    Container(

      padding: const EdgeInsets.symmetric(

        horizontal: 10,

        vertical: 6,

      ),

      decoration: BoxDecoration(

        color: statusColor.withOpacity(0.12),

        borderRadius: BorderRadius.circular(20),
```

```
    ),

    child: Text(

      statusText,

      style: TextStyle(

        color: statusColor,

        fontWeight: FontWeight.bold,

      ),

    ),

  ),

),

const SizedBox(height: 10),

DropDownButtonFormField<DeliveryStatus>(

  value: delivery.status,

  decoration: const InputDecoration(

    labelText: 'Status da entrega',

    prefixIcon: Icon(Icons.flag),

    contentPadding: EdgeInsets.symmetric(

      horizontal: 12,
```

```
        vertical: 8,

      ),

    ),

    items: const [

      DropdownMenuItem(

        value: DeliveryStatus.pending,

        child: Text('Pendente'),

      ),

      DropdownMenuItem(

        value: DeliveryStatus.inProgress,

        child: Text('Em andamento'),

      ),

      DropdownMenuItem(

        value: DeliveryStatus.completed,

        child: Text('Concluída'),

      ),

    ],
```

```
        onChanged: (value) {

            if (value != null) {

                onChangeStatus(value);

            }

        },

    ),

],

),

),

);

}

}

class ProfileScreen extends StatelessWidget {

    const ProfileScreen({super.key});

    @override

    Widget build(BuildContext context) {

        return Scaffold(

            appBar: AppBar(

                title: const Text('Perfil do Usuário'),
```

```
),

body: SingleChildScrollView(

  padding: const EdgeInsets.all(20),

  child: Column(

    children: [

      const SizedBox(height: 15),

      const CircleAvatar(

        radius: 55,

        backgroundColor: Colors.green,

        child: Icon(

          Icons.person,

          size: 65,

          color: Colors.white,

        ),

      ),

      const SizedBox(height: 15),
```

```
const Text(  
  'Estudante OrganizaIF',  
  style: TextStyle(  
    fontSize: 23,  
    fontWeight: FontWeight.bold,  
  ),  
,  
  
const SizedBox(height: 5),  
  
const Text(  
  'estudante@ifsuldeminas.edu.br',  
  style: TextStyle(  
    color: Colors.grey,  
  ),  
,  
  
const SizedBox(height: 30),  
  
Card(  

```

```
child: Column(  
  
  children: [  
  
    const ListTile(  
  
      leading: Icon(  
  
        Icons.person,  
  
        color: Colors.green,  
  
      ),  
  
      title: Text('Nome'),  
  
      subtitle: Text(  
  
        'Estudante OrganizaIF',  
  
      ),  
  
    ),  
  
    const Divider(height: 1),  
  
    const ListTile(  
  
      leading: Icon(  
  
        Icons.email,  
  
        color: Colors.green,  
  
      ),  
  
      title: Text('E-mail'),  

```

```
        subtitle: Text(

          'estudante@ifsuldeminas.edu.br',

        ),

      ),

      const Divider(height: 1),

      const ListTile(

        leading: Icon(

          Icons.school,

          color: Colors.green,

        ),

        title: Text('Instituição'),

        subtitle: Text(

          'IFSULDEMINAS',

        ),

      ),

    ],

  ),

),
```

```
const SizedBox(height: 25),

SizedBox(

  width: double.infinity,

  height: 50,

  child: OutlinedButton.icon(

    onPressed: () {

      Navigator.pushAndRemoveUntil(

        context,

        MaterialPageRoute(

          builder: (context) =>

            const LoginScreen(),

        ),

        (route) => false,

      );

    },

    icon: const Icon(Icons.logout),

    label: const Text(
```

```
        'Sair da conta',  
  
        ),  
  
        style: OutlinedButton.styleFrom(  
            foregroundColor: Colors.red,  
        ),  
    ),  
    ),  
    ),  
],  
    ),  
    ),  
);  
}
```